MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

DATABASE SYSTEMS LAB MINI PROJECT REPORT

Oracle-Based Student Performance Management System

Submitted in partial fulfillment of the requirements for the course
Database Systems Lab

Name	**Roll No.**	**Reg No.**	**Class**
Harsh Mahesh Kumar	23	240953320	CCE-A
Man Mohit Kumar	32	240953402	CCE-A

Department of Computer Science and Engineering

Manipal Institute of Technology, Manipal

Certificate

This is to certify that the mini project entitled *Oracle-Based Student Performance Management System* has been successfully completed by Harsh Mahesh Kumar (Roll No. 23, Reg No. 240953320, CCE-A) and Man Mohit Kumar (Roll No. 32, Reg No. 240953402, CCE-A) in partial fulfillment of the requirements of Database Systems Lab.

Faculty Coordinator
Department of Computer Science and Engineering
Manipal Institute of Technology, Manipal

Abstract

The project designs and implements a database-driven student performance system on Oracle DBMS. The system stores students, faculty, courses, course offerings, assessments, marks, grade cutoffs, and semester GPA records. It supports a dynamic assessment structure, so assessment components can be added or changed without redesigning the schema. Weighted totals are derived through SQL aggregation, final grades are resolved from course-specific cutoffs, and data integrity is enforced through database constraints, triggers, functions, procedures, and views. The implementation also exposes student and faculty dashboards through derived views, making the database the single source of truth for academic progress.

Contents

Certificate **i**

Abstract **ii**

1 Introduction **1**

2 Problem Statement and Objectives **2**

2.1 Problem Statement 2

2.2 Objectives 2

2.3 Functional Requirements Mapped to the Database 3

3 ER Diagram, Relation Schema, and Normalization **4**

3.1 ER Diagram 4

3.2 Final Relation Schema 5

3.3 Design Alternatives 6

3.3.1 Redundancy 6

3.3.2 Incompleteness 6

3.3.3 Decomposition 6

3.4 Normalization 7

3.4.1 First Normal Form 7

3.4.2 Second Normal Form 7

3.4.3 Third Normal Form 7

3.4.4 Boyce-Codd Normal Form 7

3.4.5 Implementation Differences from the Early Sketch 7

4 DDL Commands, PL/SQL Procedures, Functions, and Triggers **8**

4.1 Schema Definition 8

4.2 Sample Data 10

4.3 Important Queries 10

4.3.1 Compute Weighted Total Marks 10

4.3.2 Assign Grades 10

4.3.3 Class Performance Report 11

4.4 Functions and Procedures 11

4.4.1 GetGrade Function 11

4.4.2 GetTotalMarks Procedure 11

4.4.3 ClassReport Procedure 12

4.5 Triggers 12

4.5.1 Validate Assessment Weight 12

4.5.2 Validate Marks Range 13

4.6 Derived Views Used by the Application 14

5 Conclusion, Limitations, and Future Work **16**

6 References **17**

List of Tables

2.1 Requirement mapping to implemented database objects 3

CHAPTER 1

Introduction

The project is a course performance management system built around an Oracle relational schema. It models the academic workflow from course offering creation to assessment marking, final grade assignment, and semester GPA tracking. The database is not limited to a fixed number of assignments or exams; instead, every course offering can define its own set of assessment components and grade cutoffs.

The implemented application uses Oracle SQL and PL/SQL to support the following behavior:

- storing student, faculty, course, offering, assessment, attempt, attendance, cutoff, and SGPA records,
- computing weighted totals from assessment attempts,
- assigning grades from dynamically configurable cutoff ranges,
- rejecting invalid marks and invalid assessment-weight combinations at the database layer,
- generating course and class performance reports through views and procedures.

The frontend workspace uses the database through Oracle queries and derived views such as `v_student_course_progress` and `v_student_profile`. This ensures the report reflects the actual implementation rather than a purely theoretical design.

CHAPTER 2

Problem Statement and Objectives

2.1 Problem Statement

Manual tracking of course assessments, marks, and grades is error-prone when an instructor can change the evaluation pattern during the semester. A static schema with fixed columns such as `quiz1`, `quiz2`, or `midterm` cannot support courses that need a different number of assessments. The project addresses this by building a normalized schema where the assessment structure is data-driven rather than hardcoded.

2.2 Objectives

The system was designed to satisfy the following objectives:

1. store student and faculty records along with course and course-offering information,
2. support a variable number of assessment components per offering,
3. compute weighted totals using SQL aggregation,
4. keep grade cutoffs configurable at the database level,
5. persist final grades for completed enrollments,
6. enforce mark-range and weight-sum validation through database logic,
7. generate summary reports such as average, highest, and lowest scores.

2.3 Functional Requirements Mapped to the Database

Table 2.1: Requirement mapping to implemented database objects

Req.	**Implementation support**
REQ 1	`student`, `faculty`, `course`, `course_offering`, `assessment`, `grade_cutoffs`, `attends`, `attempts`, and `sgpa` tables
REQ 2	Dynamic rows in `assessment` and course-specific cutoffs in `grade_cutoffs`
REQ 3	Weighted total via aggregation in `v_student_course_progress` and helper procedures
REQ 4	Editable rows in `grade_cutoffs`
REQ 5	`GetGrade` function and persisted `final_grade` in `attends`
REQ 6	`check_marks` and `check_weight` triggers
REQ 7	`ClassReport` procedure and aggregate queries in faculty views

CHAPTER 3

ER Diagram, Relation Schema, and Normalization

3.1 ER Diagram

Figure 3.1 shows the conceptual ER diagram used for the project. The diagram includes the implementation attributes such as `password_hash`, `status`, `assessment_date`, `enrollment_date`, `attempt_date`, and `final_grade`.

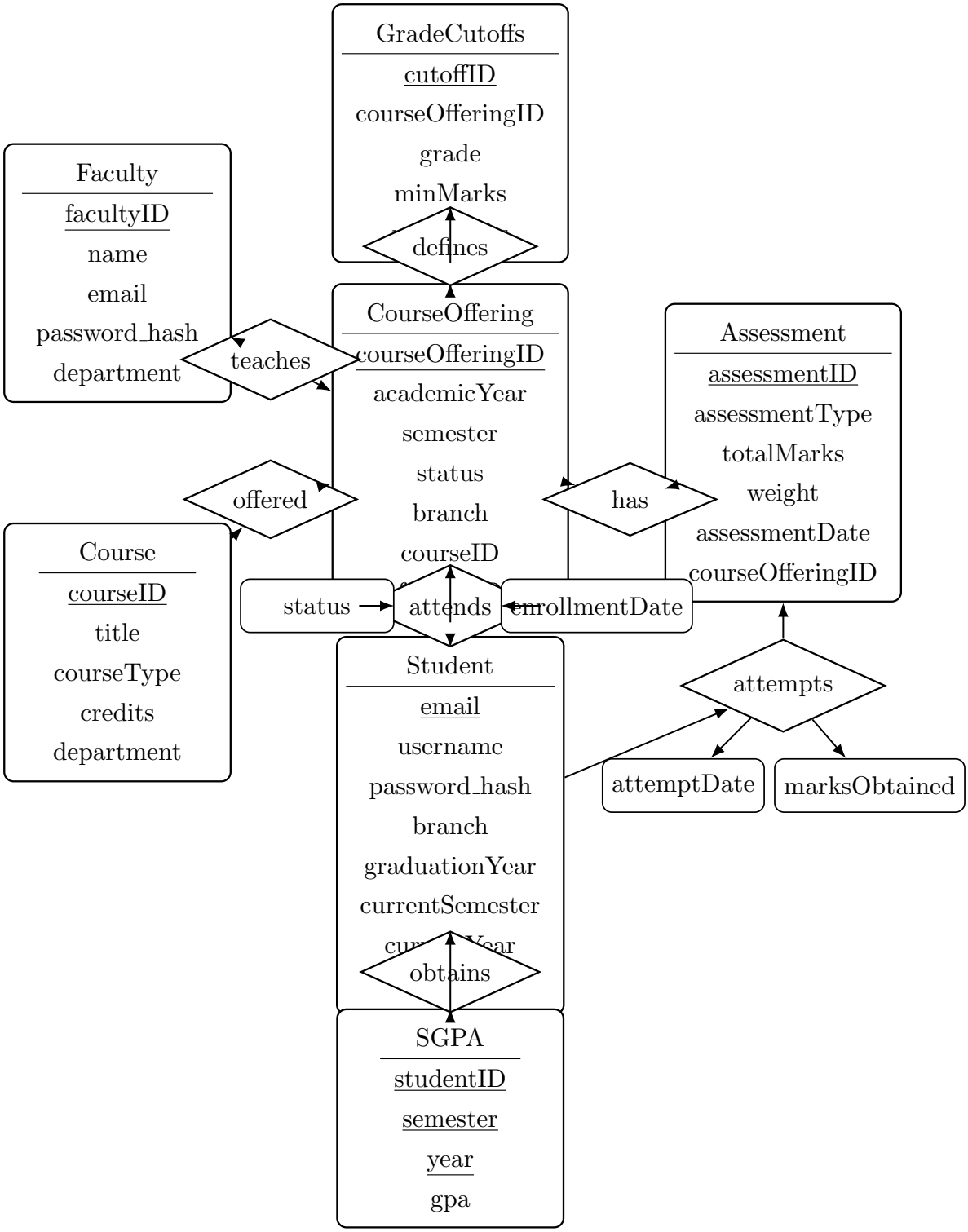

Figure 3.1: Conceptual ER diagram of the Oracle-based performance management system

3.2 Final Relation Schema

The implementation uses the following relations in Oracle:

- STUDENT(email, username, password_hash, branch, graduation_year, current_semester, current_year, role)
- FACULTY(faculty_id, name, email, password_hash, department)
- COURSE(course_id, title, course_type, credits, department)
- COURSE_OFFERING(course_offering_id, academic_year, semester, status, branch, course_id, faculty_id)
- ASSESSMENT(assessment_id, assessment_type, total_marks, weight, assessment_date, course_offering_id)
- GRADE_CUTOFFS(cutoff_id, grade, min_marks, max_marks, course_offering_id)
- SGPA(student_id, semester, academic_year, gpa)
- ATTENDS(student_mail, course_offering_id, enrollment_date, status, final_grade)
- ATTEMPTS(student_mail, assessment_id, attempt_date, marks_obtained)

3.3 Design Alternatives

3.3.1 Redundancy

An alternative design could have stored total marks, weighted totals, and final grades in separate summary tables. That design would make reads faster but would duplicate state and complicate updates. The implemented schema keeps raw academic data normalized and computes totals on demand, while storing `final_grade` in `attends` as a controlled denormalization for stable display.

3.3.2 Incompleteness

An incomplete design would omit course-offering status, cutoffs, or persistent enrollment outcomes. The implemented schema includes `status` in `course_offering` and `final_grade` in `attends`, which supports archived offerings and historical grade retrieval.

3.3.3 Decomposition

A poor design would collapse everything into one wide relation such as `student_course_data`. The current decomposition splits entities and transactions into separate tables, reducing

duplication and keeping updates local to their owning entity.

3.4 Normalization

3.4.1 First Normal Form

All attributes are atomic. There are no repeating columns such as `quiz1`, `quiz2`, or nested lists of marks. The schema is therefore in 1NF.

3.4.2 Second Normal Form

The tables with composite keys, such as `attends` and `attempts`, have attributes that depend on the full key. For example, `(student_mail, assessment_id) -> attempt_date, marks_obtained`. The schema satisfies 2NF.

3.4.3 Third Normal Form

Non-key attributes are separated into their own relations. Course details are not repeated in course offerings, and assessment details are not repeated in attempts. The schema is in 3NF, with one intentional exception: `attends.final_grade` is stored for usability.

3.4.4 Boyce-Codd Normal Form

The functional dependencies in the core relations are key-based, such as `assessment_id -> assessment_type, total_marks, weight, assessment_date, course_offering_id`. The design is BCNF-oriented, aside from the deliberate stored grade field.

3.4.5 Implementation Differences from the Early Sketch

The early ER sketch used separate relationship tables such as `HAS`, `TEACHES`, and `DEFINES`. In the final implementation, those are represented by foreign keys in `assessment`, `course_offering`, and `grade_cutoffs`. The codebase also includes derived views that were not visible in the sketch, namely `v_student_course_progress` and `v_student_profile`.

CHAPTER 4

DDL Commands, PL/SQL Procedures, Functions, and Triggers

4.1 Schema Definition

The following listing summarizes the Oracle schema used by the project.

Listing 4.1: Core DDL commands

```
CREATE TABLE student (
  email VARCHAR2(255) PRIMARY KEY,
  username VARCHAR2(120) NOT NULL,
  password_hash VARCHAR2(255) NOT NULL,
  branch VARCHAR2(20) NOT NULL,
  graduation_year NUMBER(4) NOT NULL,
  current_semester NUMBER(2) NOT NULL,
  current_year NUMBER(4) NOT NULL,
  role VARCHAR2(20) DEFAULT 'student' NOT NULL
);

CREATE TABLE faculty (
  faculty_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name VARCHAR2(120) NOT NULL,
  email VARCHAR2(255) NOT NULL UNIQUE,
  password_hash VARCHAR2(255) NOT NULL,
  department VARCHAR2(20) NOT NULL
);

CREATE TABLE course_offering (
  course_offering_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  academic_year NUMBER(4) NOT NULL,
  semester NUMBER(2) NOT NULL,
```

```
  status VARCHAR2(20) DEFAULT 'current' NOT NULL,
  branch VARCHAR2(20) NOT NULL,
  course_id VARCHAR2(20) NOT NULL,
  faculty_id NUMBER NOT NULL,
  CONSTRAINT fk_course_offering_course FOREIGN KEY (course_id) REFERENCES
    course(course_id),
  CONSTRAINT fk_course_offering_faculty FOREIGN KEY (faculty_id) REFERENCES
    faculty(faculty_id)
);

CREATE TABLE assessment (
  assessment_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  assessment_type VARCHAR2(60) NOT NULL,
  total_marks NUMBER(4) NOT NULL,
  weight NUMBER(5,2) NOT NULL,
  assessment_date DATE DEFAULT SYSDATE NOT NULL,
  course_offering_id NUMBER NOT NULL,
  CONSTRAINT fk_assessment_course_offering FOREIGN KEY (course_offering_id)
    REFERENCES course_offering(course_offering_id)
);

CREATE TABLE grade_cutoffs (
  cutoff_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  grade VARCHAR2(3) NOT NULL,
  min_marks NUMBER(5,2) NOT NULL,
  max_marks NUMBER(5,2) NOT NULL,
  course_offering_id NUMBER NOT NULL,
  CONSTRAINT fk_cutoff_course_offering FOREIGN KEY (course_offering_id)
    REFERENCES course_offering(course_offering_id)
);

CREATE TABLE attends (
  student_mail VARCHAR2(255) NOT NULL,
  course_offering_id NUMBER NOT NULL,
  enrollment_date DATE NOT NULL,
  status VARCHAR2(20) NOT NULL,
  final_grade VARCHAR2(3),
  CONSTRAINT pk_attends PRIMARY KEY (student_mail, course_offering_id),
  CONSTRAINT fk_attends_student FOREIGN KEY (student_mail) REFERENCES student(
    email),
  CONSTRAINT fk_attends_course_offering FOREIGN KEY (course_offering_id)
```

```
    REFERENCES course_offering(course_offering_id)
);
```

4.2 Sample Data

The seeded dataset in the workspace follows the same structure and can be reproduced with the reset script. The data includes multiple faculty members, students across branches, course offerings for different semesters, assessments with varying weights, and attempts for enrolled students.

4.3 Important Queries

4.3.1 Compute Weighted Total Marks

```
SELECT
  a.student_mail,
  SUM((a.marks_obtained / b.total_marks) * b.weight) AS total_score
FROM attempts a
JOIN assessment b ON a.assessment_id = b.assessment_id
GROUP BY a.student_mail;
```

4.3.2 Assign Grades

```
SELECT
  t.student_mail,
  t.total_score,
  g.grade
FROM (
  SELECT
    a.student_mail,
    SUM((a.marks_obtained / b.total_marks) * b.weight) AS total_score
  FROM attempts a
  JOIN assessment b ON a.assessment_id = b.assessment_id
  GROUP BY a.student_mail
) t
JOIN grade_cutoffs g
  ON t.total_score BETWEEN g.min_marks AND g.max_marks;
```

4.3.3 Class Performance Report

```
SELECT
  AVG(marks_obtained) AS average,
  MAX(marks_obtained) AS highest,
  MIN(marks_obtained) AS lowest
FROM attempts;
```

4.4 Functions and Procedures

4.4.1 GetGrade Function

```
CREATE OR REPLACE FUNCTION GetGrade(
  p_score NUMBER,
  p_course_offering_id NUMBER
) RETURN VARCHAR2
IS
  v_grade VARCHAR2(3);
BEGIN
  SELECT grade
    INTO v_grade
    FROM grade_cutoffs
   WHERE p_score BETWEEN min_marks AND max_marks
     AND course_offering_id = p_course_offering_id
   ORDER BY min_marks DESC
   FETCH FIRST 1 ROW ONLY;

  RETURN v_grade;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 'F';
END;
```

4.4.2 GetTotalMarks Procedure

```
CREATE OR REPLACE PROCEDURE GetTotalMarks(
  p_student IN VARCHAR2,
  p_total OUT NUMBER
```

```
)
IS
BEGIN
  SELECT NVL(ROUND(SUM((at.marks_obtained / NULLIF(asmt.total_marks, 0)) *
   asmt.weight), 2), 0)
    INTO p_total
    FROM attempts at
    JOIN assessment asmt ON asmt.assessment_id = at.assessment_id
   WHERE at.student_mail = p_student;
END;
/
```

4.4.3 ClassReport Procedure

```
CREATE OR REPLACE PROCEDURE ClassReport(
  p_course_offering_id IN NUMBER,
  p_average OUT NUMBER,
  p_highest OUT NUMBER,
  p_lowest OUT NUMBER
)
IS
BEGIN
  SELECT NVL(ROUND(AVG(at.marks_obtained), 2), 0),
         NVL(MAX(at.marks_obtained), 0),
         NVL(MIN(at.marks_obtained), 0)
    INTO p_average, p_highest, p_lowest
    FROM attempts at
    JOIN assessment asmt ON asmt.assessment_id = at.assessment_id
   WHERE asmt.course_offering_id = p_course_offering_id;
END;
/
```

4.5 Triggers

4.5.1 Validate Assessment Weight

```
CREATE OR REPLACE TRIGGER check_weight
FOR INSERT OR UPDATE ON assessment
```

```
COMPOUND TRIGGER
  TYPE t_course_ids IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  g_course_ids t_course_ids;
  g_index PLS_INTEGER := 0;

  BEFORE EACH ROW IS
  BEGIN
    g_index := g_index + 1;
    g_course_ids(g_index) := :NEW.course_offering_id;
  END BEFORE EACH ROW;

  AFTER STATEMENT IS
    v_total NUMBER;
  BEGIN
    FOR i IN 1 .. g_index LOOP
      SELECT NVL(SUM(weight), 0)
        INTO v_total
        FROM assessment
       WHERE course_offering_id = g_course_ids(i);

      IF v_total > 100 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Total assessment weight exceeds 100')
    ;
      END IF;
    END LOOP;
  END AFTER STATEMENT;
END;
/
```

4.5.2 Validate Marks Range

```
CREATE OR REPLACE TRIGGER check_marks
BEFORE INSERT OR UPDATE ON attempts
FOR EACH ROW
DECLARE
  v_max NUMBER;
BEGIN
  SELECT total_marks
    INTO v_max
    FROM assessment
```

```
   WHERE assessment_id = :NEW.assessment_id;

  IF :NEW.marks_obtained < 0 OR :NEW.marks_obtained > v_max THEN
    RAISE_APPLICATION_ERROR(-20001, 'Marks exceed the maximum allowed for this
     assessment');
  END IF;
END;
/
```

4.6 Derived Views Used by the Application

The application reads student progress through views rather than computing every presentation query in the UI.

```
CREATE OR REPLACE VIEW v_student_course_progress AS
SELECT
  a.student_mail,
  fo.course_offering_id,
  fo.academic_year,
  fo.semester,
  fo.branch,
  c.course_id,
  c.title AS course_title,
  c.course_type,
  c.credits,
  f.name AS faculty_name,
  NVL((
    SELECT ROUND(SUM(at.marks_obtained), 2)
    FROM attempts at
    JOIN assessment asmt ON asmt.assessment_id = at.assessment_id
    WHERE at.student_mail = a.student_mail
      AND asmt.course_offering_id = fo.course_offering_id
  ), 0) AS earned_raw,
  NVL((
    SELECT ROUND(SUM(asmt.total_marks), 2)
    FROM attempts at
    JOIN assessment asmt ON asmt.assessment_id = at.assessment_id
    WHERE at.student_mail = a.student_mail
      AND asmt.course_offering_id = fo.course_offering_id
  ), 0) AS possible_raw,
```

```
  NVL((
    SELECT ROUND(SUM((at.marks_obtained / NULLIF(asmt.total_marks, 0)) * asmt.
   weight), 2)
    FROM attempts at
    JOIN assessment asmt ON asmt.assessment_id = at.assessment_id
    WHERE at.student_mail = a.student_mail
      AND asmt.course_offering_id = fo.course_offering_id
  ), 0) AS weighted_total,
  a.final_grade,
  a.status AS status
FROM attends a
JOIN course_offering fo ON fo.course_offering_id = a.course_offering_id
JOIN course c ON c.course_id = fo.course_id
JOIN faculty f ON f.faculty_id = fo.faculty_id;
```

CHAPTER 5

Conclusion, Limitations, and Future Work

The project demonstrates how Oracle SQL and PL/SQL can support a realistic academic workflow without hardcoded assessment columns. The implemented schema satisfies the main requirements for dynamic assessments, computed totals, grading cutoffs, data validation, and report generation.

The primary limitation is that the current implementation stores one attempt per student per assessment and relies on application logic to refresh final grades after changes. A future enhancement could add multi-attempt support, audit history for cutoff changes, and automatic refresh mechanisms for derived grade summaries. Another useful extension would be a richer analytics layer for section-wise performance comparison and transcript generation.

CHAPTER 6

References

Bibliography

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, Pearson.

[2] Oracle Corporation, *Oracle Database SQL Language Reference*.

[3] Oracle Corporation, *PL/SQL Language Reference*.

[4] Next.js Documentation, *https://nextjs.org/docs*.